

第3章

指令系统与程序设计



3.2 C语言程序设计

- 3.1 指令系统与寻址方式
- 3.2 C语言程序设计



3.1.1 微处理器指令系统

- 3.1.1 微处理器指令系统
- 3.1.2 MSP430单片机指令集
- 3.1.3 MSP430寻址方式与指令周期
- 3.1.4 鲲鹏920处理器的架构及寻址



3.1.1 微处理器指令系统

● 指令

- ◆ 完成特定操作的命令称为**指令**。
- ◆ 指出**数据的来源、操作结果的去向及所执行的操作**。
- ◆ 一条指令通常由**操作码**和**操作数**组成。

学习指令有助于了解：计算机是怎么工作的

CPU设计是针对指令处理展开的。
厂家总是不遗余力的公开指令集
反之就是块石头，没有使用价值



3.1.1 微处理器指令系统

● 操作码

- ◆ 具体说明操作的性质及功能。
- ◆ 每一条指令都有一个相应的操作码，计算机通过识别该操作码来完成不同的操作。
- ◆ 其长度取决于指令系统中的指令条数。

● 操作数

- ◆ 在运行过程需要的数据称为操作数。
- ◆ 直接给出操作数或指出操作数的存储器地址或寄存器地址（即寄存器名）。



3.1.1 微处理器指令系统

- 由于指令系统是基于汇编语言实现的，故亦称为汇编语言指令书写格式

[标号:]	操作码	操作数	[; 注释]
-------	-----	-----	--------

例如

Label: MOV R0, R5 ; R0→R5



3.1.1 微处理器指令系统

“最小版本” CPU指令集为例，指令 vs CPU控制器

		$I_7 I_6 I_5 I_4$	$I_3 I_2$	$I_1 I_0$
NOP	} 单字节指令:	0 0 0 0	X X	0 0/1
HLT				
JMP	双字节指令:	0 0 0 1	0 0	0 0
		addr		

汇编语言	功能	$I_7 I_6 I_5 I_4$	$I_3 I_2$	$I_1 I_0$
NOP	空指令：不执行任何操作	0 0 0 0	X X	0 0
HLT	“断点”：硬件直接停机	0 0 0 0	X X	X 1
JMP ADDR	直接寻址：ADDR → PC	0 0 0 1	0 0	0 0
		ADDR		



3.1.1 微处理器指令系统

机器指令 vs 汇编指令
程序示例

汇编助记符	注释	地址：机器指令
JMP, 03H	程序跳转到地址03H执行 03H→PC	00H:00010000
		01H:00000011
NOP	空	02H:00000000
HLT	停机	03H:00000001
JMP, 02H	程序跳转到地址02H执行 02H→PC	04H:00010000
		05H:00000010



3.1.2 MSP430单片机指令集

- 3.1.1 微处理器指令系统
- **3.1.2 MSP430单片机指令集**
- 3.1.3 MSP430寻址方式与指令周期
- 3.1.4 鲲鹏920处理器的架构及寻址



3.1.2 MSP430单片机指令集

- MSP430的指令系统是16位精简指令系统，共有**51条指令**，其中**27条为内核指令**（core instruction,有自己的操作码），**24条为仿真指令**（emulated instruction,无自己的操作码，作用是便于程序员读写；汇编时，汇编器把仿真指令转换成具有唯一操作码的内核指令，因此，仿真指令的使用并不会影响指令长度和执行效率。）
- 该指令系统具有**寻址方式多、执行速度快、功能强的优点**。



3. 1. 2 MSP430单片机指令集

- MSP430的指令系统按其**功能**划分，可分为：
 - ◆ 数据传送指令(**4**+**2**条)
 - ◆ 算术运算指令(**6**+**8**条)
 - ◆ 逻辑操作指令(**7**+**3**条)
 - ◆ 位操作指令(**0**+**8**条)
 - ◆ 控制转移指令(**10**+**3**条)



3. 1. 2 MSP430单片机指令集

1.数据传送指令

- **MOV[.B] src, dst ; src→dst**
- **SWPB dst ; 第15-8位与第7-0位互换**
- **SXT dst ; 将第7位填充到第8-15位中**
- **PUSH[.B] src ; SP-2→SP, src→(SP)**
- ***POP[.B] dst ; (SP) →src, SP+2→SP**
- ***CLR[.B] dst ; 0→dst**

.B表示字节指令，即以字节方式访问数据或外设。

.W表示字指令，以字方式访问数据或外设。

不使用后缀，系统默认为字指令。



3. 1. 2 MSP430单片机指令集

2. 算术运算指令

- MSP430的算术运算指令主要包括
 - ◆ 加法运算指令 (3+4)
 - ◆ 减法运算指令 (3+4)
- 并且这两类指令均会对标志位产生相应的影响。但对控制标志位(OSCOFF、CPUOFF、GIE)不产生任何影响。



3. 1. 2 MSP430单片机指令集

加法运算指令

ADD[.B]	src, dst	; src+dst→dst
ADDC[.B]	src, dst	; src+dst+C→dst
DADD[.B]	src, dst	; src+dst+C→dst(十进制)
ADC[.B]	dst	; dst+C→dst
DADC[.B]	dst	; dst+C→dst(十进制)
INC[.B]	dst	; dst+1→dst
INCD[.B]	dst	; dst+2→dst



3. 1. 2 MSP430单片机指令集

减法运算指令

SUB[.B]	src, dst	; dst-src $\rightarrow$ dst
SUBC[.B]	src, dst	; dst-src-1+C $\rightarrow$ dst
CMP[.B]	src, dst	; dst-src
SBC[.B]	dst	; dst-1+C $\rightarrow$ dst
TST[.B]	dst	; dst-0
DEC[.B]	dst	; dst-1 $\rightarrow$ dst
DECD[.B]	dst	; dst-2 $\rightarrow$ dst



3.1.2 MSP430单片机指令集

3.逻辑操作指令

- 逻辑操作指令共10条，
 - ◆ 逻辑运算指令6条，
 - ◆ 逻辑移位指令4条。
- 这些指令对控制标志位(OSCOFF、CPUOFF、GIE)不产生任何影响，部分指令对状态标志位有影响。



3. 1. 2 MSP430单片机指令集

6条逻辑运算指令：

- AND 源操作数和目的操作数与 AND.B src,dst /AND[.W] src,dst
- BIC 清除目的操作数的各位 BIC.B src,dst /BIC [.W] src,dst
- BIS 设置目的操作数的各位 BIS.B src,dst /BIS[.W] src,dst
- BIT 测试目的操作数的各位 BIT.B src,dst /BIT[.W] src,dst
- XOR 源操作数和目的操作数异或 XOR.B src,dst /XOR[.W] src,dst
- INV 目的操作数求反 INV.B dst /INV[.W] dst

AND、XOR、INV、BIT指令对状态标志位有影响；**BIC、BIS**对状态标志位不产生任何影响



3.1.2 MSP430单片机指令集

4条逻辑移位指令：

RLA	算数左移	RLA.B dst /RLA[.W] dst
RLC	通过进位位左移	RLC.B dst /RLC[.W] dst
RRA	算数右移	RRA.B dst /RRA[.W] dst
RRC	通过进位位右移	RRC.B dst /RRC[.W] dst





3. 1. 2 MSP430单片机指令集

4.位操作指令

- 此类指令是无操作数指令，只能对状态寄存器中的位进行操作，亦称为状态位操作指令。
- 除了状态寄存器之外，其他的MSP430寄存器是不具备位寻址的。
- 要想实现位操作，只能通过字或字节掩膜的办法实现。



3. 1. 2 MSP430单片机指令集

- *CLRC 进位位清零
- *SETC 进位位置1
- *CLRZ 零标志位清零
- *SETZ 零标志位置1
- *CLRNC 负标志位清零
- *SETN 负标志位置1
- *EINT 开总中断
- *DINT 关总中断



3. 1. 2 MSP430单片机指令集

5. 控制转移指令

- 此类指令大致可分为
 - ◆ 无条件转移
 - ◆ 条件转移
 - ◆ 子程序调用
 - ◆ 返回指令
 - ◆ 空操作指令。
- 该类指令(除RETI外)对状态标志位不产生任何影响。



3. 1. 2 MSP430单片机指令集

BR	转移至目的操作数	BR dst
CALL	调用子程序	CALL dst
JC/JHS	C=1时程序跳转 大于和等于时程序跳转	JC 标号 JHS 标号
JEQ/JZ	等于时程序跳转 Z=1时程序跳转	JEQ 标号 JZ 标号
JGE	大于等于时程序跳转	JGE 标号
JL	小于时程序跳转	JL 标号
JMP	程序无条件跳转	JMP 标号
JN	为负时程序跳转	JN 标号
JNC/JLO	C=0时程序跳转 小于时程序跳转	JNC 标号 JLO 标号
JNE/JNZ	不等时程序跳转 不为零时程序跳转	JNE 标号 JNZ 标号
RET	从子程序返回	RET
RETI	从中断返回	RETI
NOP	空操作	NOP



3.1.3 MSP430寻址方式与指令周期

- 3.1.1 微处理器指令系统
- 3.1.2 MSP430单片机指令集
- 3.1.3 MSP430寻址方式与指令周期
- 3.1.4 鲲鹏920处理器的架构及寻址



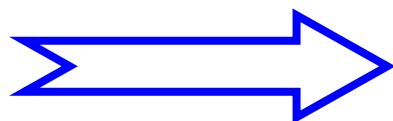
3. 1. 3 MSP430寻址方式与指令周期

- 单片机执行程序的过程，实际上就是不断寻找操作数并对操作数进行操作的过程。
- 寻址方式：在程序执行过程中，指令寻找操作数所使用的方法。
- 一般情况下，寻址方式越多，功能就越强，灵活性也就越大。

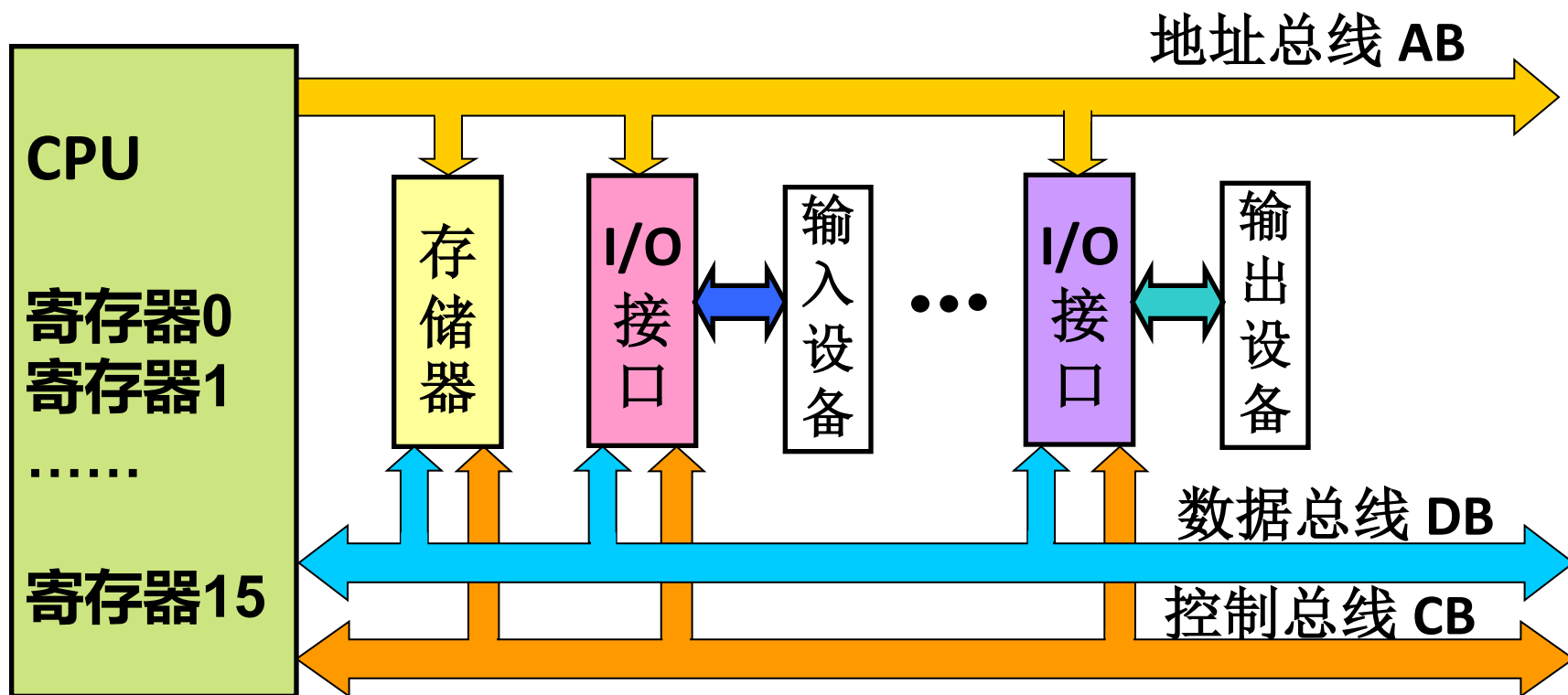


3.1.3 MSP430寻址方式与指令周期

在汇编语言中需指明
操作数所在的地方

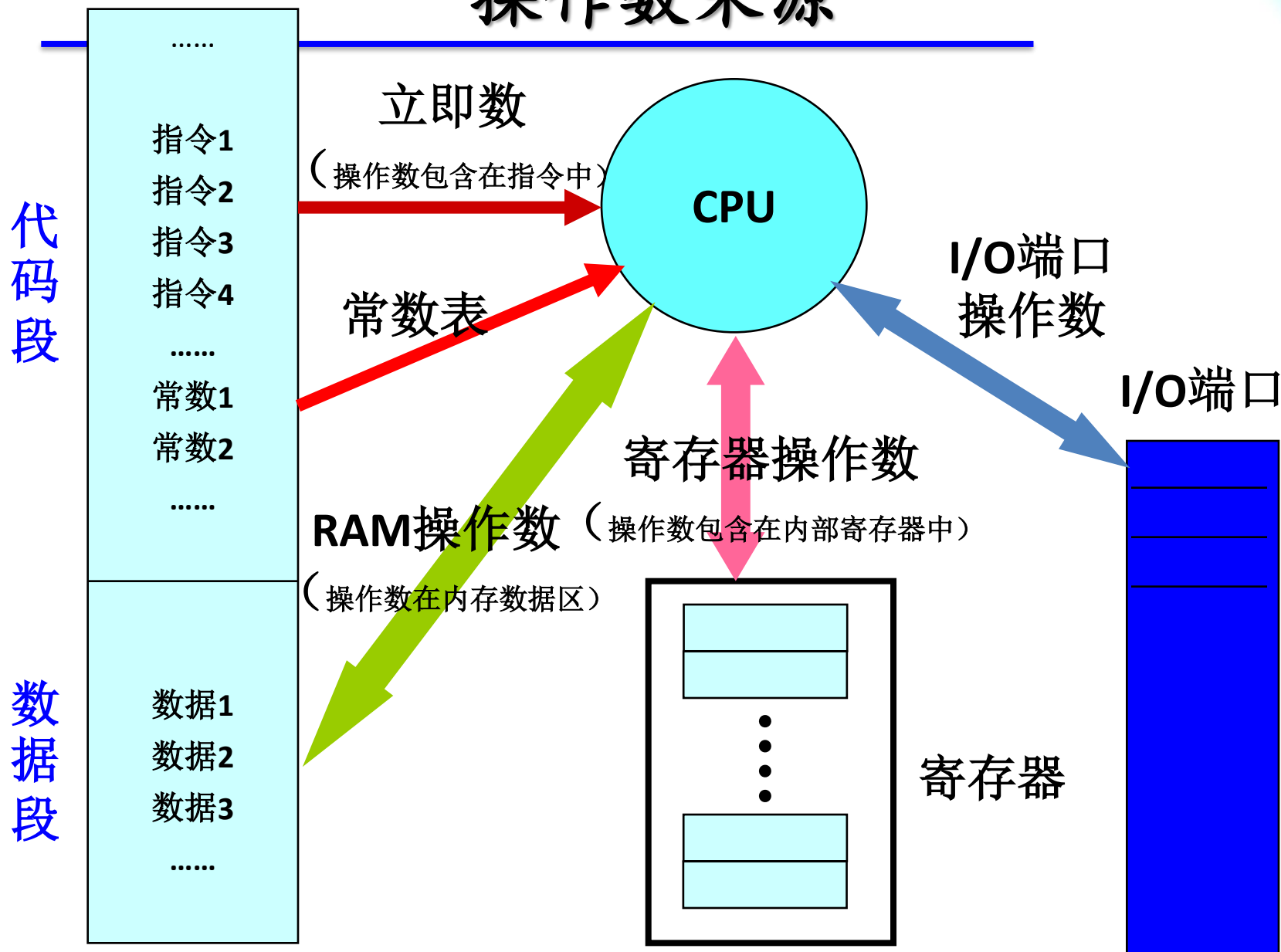


寻址方式





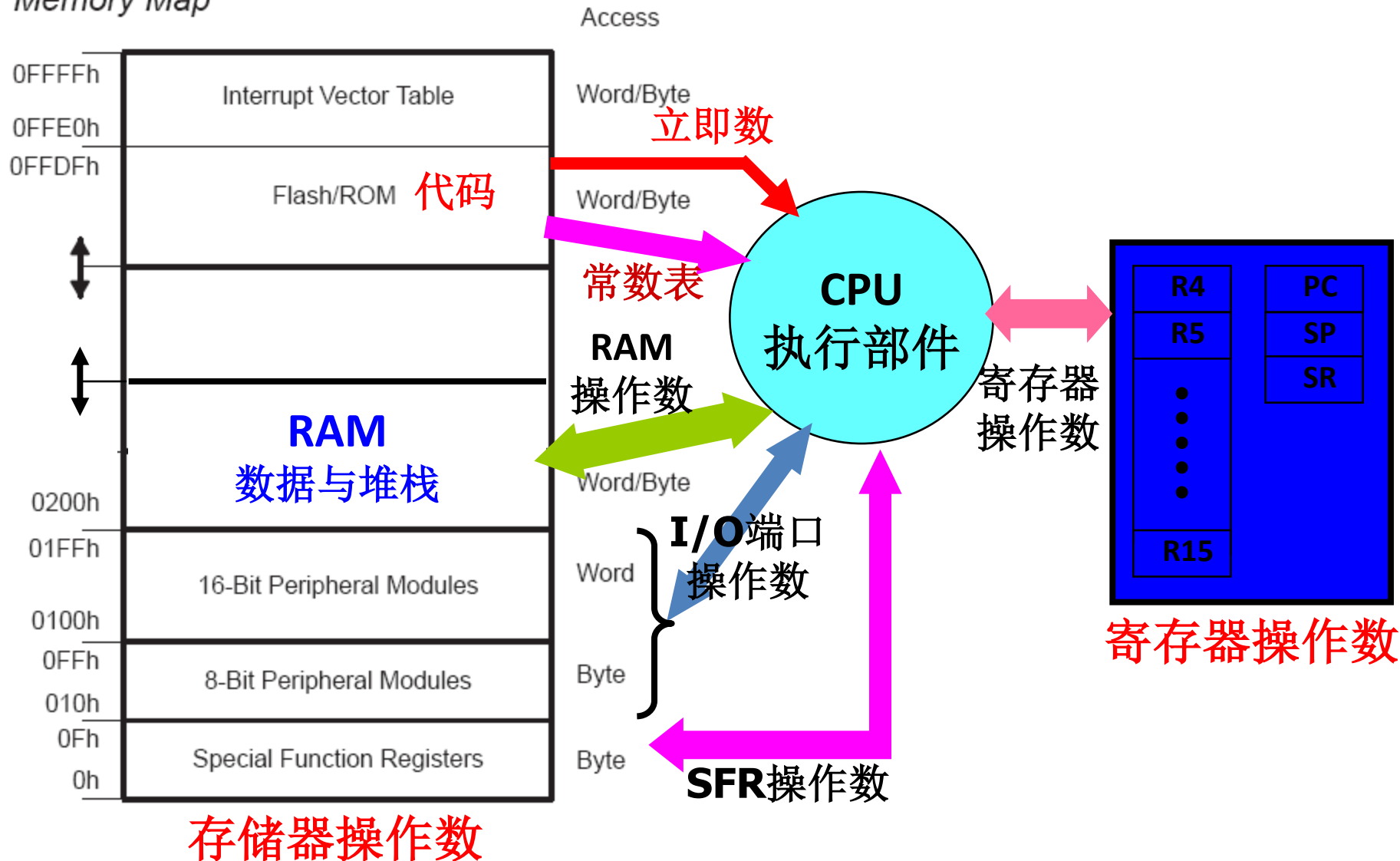
操作数来源



MSP430 操作数的来源示意图



Memory Map





3. 1. 3 MSP430寻址方式与指令周期

MSP430单片机支持寄存器寻址、变址寻址、符号寻址、绝对寻址、寄存器间接寻址、自动增量间接寻址、立即寻址等7种寻址方式。



3. 1. 3 MSP430寻址方式与指令周期

以 MOV 指令为例：MOV src , dst
(src) → (dst)

- | | |
|--------------|----------------------|
| 1. 寄存器寻址 | 例：MOV R4 , R5 |
| 2. 变址寻址 | 例：MOV R5 , x(R4) |
| 3. 符号寻址 | 例：MOV R6 , label |
| 4. 绝对寻址 | 例：MOV R7 , &variable |
| 5. 寄存器间接寻址 | 例：MOV @R9 , R4 |
| 6. 寄存器间接增量寻址 | 例：MOV @R5+ , R6 |
| 7. 立即数寻址 | 例：MOV #1000h , R4 |

任一操作数都有明确的寻址方式



3.1.3 寻址方式--立即寻址

- 又称立即数寻址
- 在这种寻址方式中，操作数在指令中由源操作数直接给出
- 例如 `MOV #0020H, R4 ; 20H→R4`

注意：1. 立即数前需要加“#”，即#data。否则将会出错。

例如 `MOV 0020H, R4 ;`为非法指令

2. 立即寻址只能用于源操作数的寻址。

例如 `MOV R4, #0020H` 则是错误的用法。



3.1.3 寻址方式--绝对寻址

- 又称直接寻址
- 绝对寻址的操作数位于RAM内，操作数的地址直接在指令中给出

例如 `MOV R9, &2360H ; R9→(2360)`

`MOV &2000H, R7 ; (2000)→R7`

注意：

绝对地址前需要加“&”，即&addr。否则将会出错。

例如 `MOV 2000H, R7 ;为非法指令`

- 绝对寻址方式适用于源操作数寻址和目的操作数寻址。
- 该寻址方式主要用于访问具有绝对固定地址的外设模块。对他们使用绝对寻址，可以保证软件的透明度。



3.1.3 寻址方式--符号寻址

- 该寻址方式的操作数在RAM中，但操作数的地址却在指令中以**标号的形式**给出。

例如 **AAA** MOV #2036H, R9 ; **AAA**表示该指令的起始地址
MOV AAA, R8 ; 将AAA地址单元的内容存放在R8

注意

- 1) 这里所说的符号是程序员在程序中自己定义的标号，其代表的是此**指令的起始地址**。
- 2) 符号寻址中，符号所指代的具体地址可**由汇编器计算得到**，也可由程序员给出；而绝对寻址中的地址是由**程序员在编写程序时**给出的。
- 3) 标号前加上“&”，表示的是地址。
- 4) 该寻址方式适用于源操作数寻址和目的操作数寻址。
- 5) 该寻址方式一般用于随机访问。



3.1.3 寻址方式--符号寻址

又如

MOVE Tab1, R10 ;将数据表中第一个字存放在R10中
Tab1 DW 10E2H, 2214H, 59A2H; 定义一块数据区, 标号
; Tab1代表该区域的起始地址

该语句执行后的结果为R10 = ?

R10 = 10E2H



3.1.3 寻址方式--寄存器寻址

- 该寻址方式的操作数存放在寄存器中。

例如

MOV R10, R8 ; R10→R8

注意

- 1) 该方式将源操作中内容移动到目的操作数，但源操作数的内容不变。
- 2) 该方式适用于源操作数寻址和目的操作数寻址。
- 3) 该方式通常用于对时间要求较严格的操作。



3.1.3 寻址方式--变址寻址

- 该寻址方式的操作数存放在RAM中，操作数的地址为寄存器的内容加上前面的偏移量。

例如

MOV #0400H, R7 ; 0400H→R7

MOV #0530H, R4 ; 0530H→R4

MOV 3(R7), 5(R4) ; (0400H+3)→(0530H+5)

注意

- 1) 该方式源操作数或目的操作数涉及的寄存器内容在执行前后不变。
- 2) 该方式适用于源操作数寻址和目的操作数寻址。



3.1.3 寻址方式--寄存器间接寻址

- 该寻址方式的操作数存放在RAM中，但此操作数的地址位于寄存器中。

例如

MOV #0453H, R5 ; 0453H → R5

MOV @R5, R4 ; (0453H) → R4

注意：

- ◆ 该寻址方式只适用于源操作数。
- ◆ 若要在目的操作数中也使用间址寻址，需用变址寻址方式间接实现，即用0(Rn)代替@Rn。

例如 MOV @R5, 0(R4)即可实现目的操作数的寄存器间接寻址。



3.1.3 寻址方式--自动增量寄存器间接寻址

- 该寻址方式的操作数存放在RAM中，但此操作数的地址位于寄存器中。
- 可以看出该寻址方式与寄存器间接寻址方式大致相同，
- 唯一不同的是，使用自动增量寄存器间接寻址方式的寄存器在执行完间接寻址之后，会自动增加。
- 具体增加的数量视指令类型而定。如对于字节指令，自动加1；对于字指令自动加2。



3.1.3 寻址方式--自动增量寄存器间接寻址

● 例如

MOV #0201H, R8 ; 0201H→R8

MOV #3000H, R7 ; 3000H→R7

**MOV @R8+, 0(R7) ; (0201H)→(3000H),
; 然后 R8=R8+2**

注意

- 1) 该寻址方式只适用于源操作数。
- 2) 该寻址方式适于对表进行随机访问，但目的寄存器的内容需程序员手动改变。



3.1.3 寻址方式

MSP430的源操作数可以使用上述7种寻址方式，而目的操作数只能用其中4种寻址方式。

寻址方式	使用场合	
寄存器寻址	源操作数	目的操作数
变址寻址	源操作数	目的操作数
符号寻址	源操作数	目的操作数
绝对寻址	源操作数	目的操作数
间接寻址	源操作数	
自动增量寄存器间接寻址	源操作数	
立即寻址	源操作数	



3. 1. 3 MSP430寻址方式与指令周期

- 指令周期是执行一条指令所需要的时间，即从取指令、分析指令到执行完所需的全部时间。
- 可见，指令周期就是指令的执行周期，亦称指令执行周期。
- 指令的执行周期取决于指令字格式和寻址方式。
- 通常，指令的执行周期是以主时钟(MCLK)做为参考的。



3. 1. 3 MSP430寻址方式与指令周期

1. 寻址方式对指令周期的影响

- 同一条指令的指令周期会因所采用的寻址方式不同而存在较大的差异
 - 1) 寄存器组之间的数据传输：执行效率最高，1个MCLK
 - 2) 寄存器与立即数之间的数据传输(如，源操作数为立即数寻址，目的操作数为寄存器寻址)：效率次之，2个MCLK
 - 3) 寄存器与存储器之间的数据传输（如，源操作数为寄存器寻址，目的操作数为符号寻址）：效率低，4个MCLK
 - 4) 存储器与存储器之间的数据传输（如，源操作数为自动增量间接寻址，目的操作数为符号寻址）：效率最低，5个MCLK



3. 1. 3 MSP430寻址方式与指令周期

2. 指令格式对指令周期的影响

- 影响指令周期长短的主要因素是寻址方式。但指令中操作数的多少也对指令周期产生一定影响。
- 对于跳转指令而言，其指令周期是固定的，均是2个MCLK。
- 通常单操作数指令一般要比双操作数指令要快。



3. 1. 3 MSP430寻址方式与指令周期

练习：指出下面指令中源操作数的寻址方式。

- | | |
|----------------------|-----------------|
| (1) MOV R4, R8 | (1) 寄存器寻址 |
| (2) MOV &2036H, R9 | (2) 绝对寻址 |
| (3) MOV 2(R5), 7(R6) | (3) 变址寻址 |
| (4) MOV @R5, R4 | (4) 寄存器间接寻址 |
| (5) MOV @R5+, 5(R3) | (5) 自动增量寄存器间接寻址 |
| (6) MOV #2011, R9 | (6) 立即寻址 |



3.1.4 鲲鹏920处理器的架构及寻址

- 3.1.1 微处理器指令系统
- 3.1.2 MSP430单片机指令集
- 3.1.3 MSP430寻址方式与指令周期
- 3.1.4 鲲鹏920处理器的架构及寻址

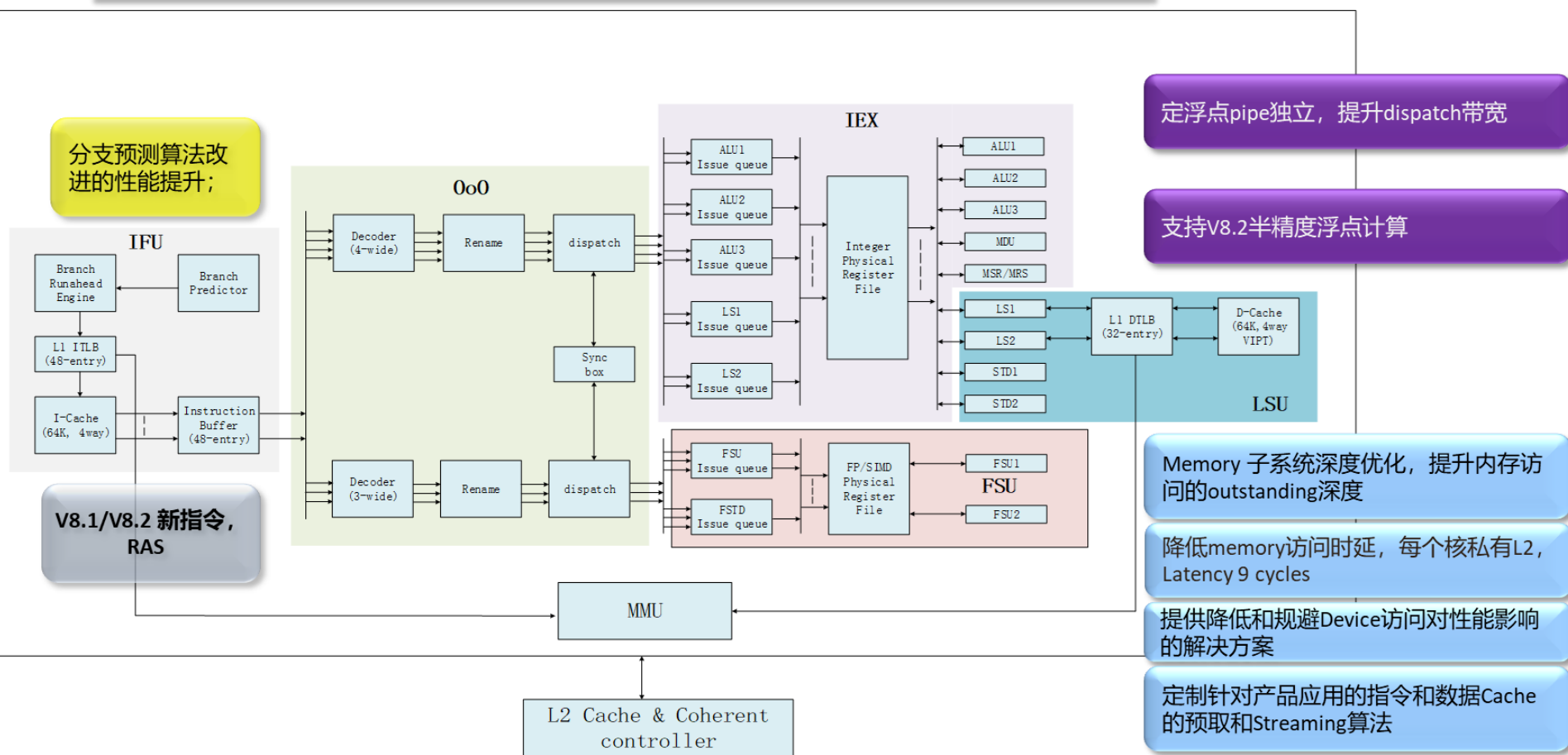


视频

- 5 鲲鹏920处理器的指令寻址方式
- 6 鲲鹏920处理器的指令寻址方式举例



3.1.4 鲲鹏920处理器的架构及寻址



基于ARMv8的鲲鹏处理器微架构

*MMU: Memory Management Unit, 内存管理单元, 负责处理CPU的内存访问请求, 功能包括虚拟地址到物理地址的转换、内存保护、中央处理器高速缓存的控制等。



3.1.4 鲲鹏920处理器的架构及寻址

ARMv8-A的指令集

2011年，ARM公司发布首个支持64位指令集的新一代ARMv8处理器架构，向下兼容ARMv7。

两套指令集：工作寄存器的位数不同

- A64指令集：ARMv8-A的64位指令集
- A32指令集：ARMv7指令集，即之前的ARM指令集



3.1.4 鲲鹏920处理器的架构及寻址

ARMv8-A的执行状态

执行状态：处理单元的执行环境，包括其所支持的寄存器宽度、支持的指令集、异常模型、虚拟存储系统体系结构和编程模型等主要特征。

AArch64执行状态

- 支持单一的A64指令集
- 放弃了传统ARM处理器的工作模式、异常模式等概念，转而定义了4个异常等级（EL0~EL3）
- 支持48-52位物理地址
- 支持31个64位通用寄存器X0-X30，64位PC



3.1.4 鲲鹏920处理器的架构及寻址

ARMv8-A的执行状态

AArch32执行状态

- 支持A32和T32（Thumb-2）指令集
- 支持13个32位通用寄存器、32位PC、堆栈指针SP和链接寄存器LR

鲲鹏处理器内置的Taishan V110处理器仅支持AArch64
执行状态



3.1.4 鲲鹏920处理器的架构及寻址

ARMv8-A支持的数据类型

- 8位字节、16位半字、32位字、64位双字
- 128位的四字（Quadword）数据类型
- 半精度浮点数据、单精度浮点数据、双精度浮点数据
- 字型和双字型定点数和向量类型



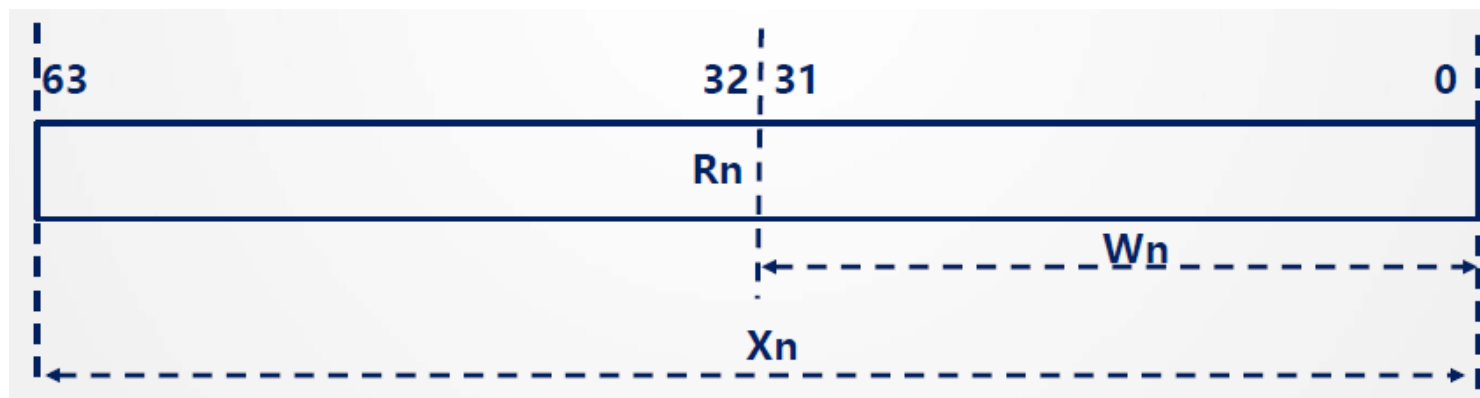
3.1.4 鲲鹏920处理器的架构及寻址

ARMv8-A的寄存器

通用寄存器：AArch64状态下支持31个通用寄存器R0-R30，每个寄存器都可以当做64位或32位寄存器使用

- 64位访问：X0-X30

- 32位访问：W0-W30

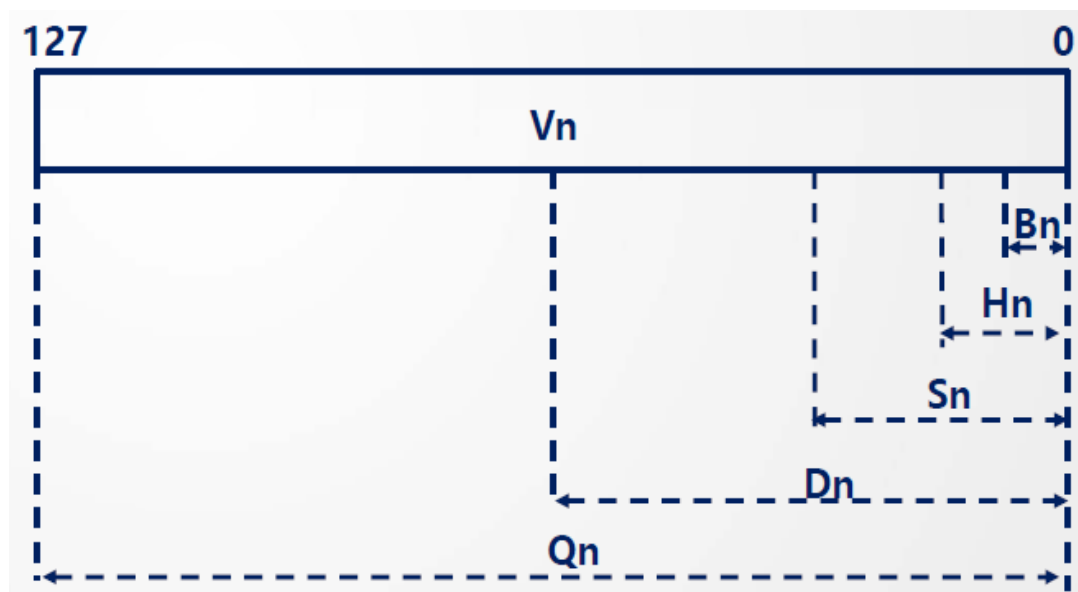




3.1.4 鲲鹏920处理器的架构及寻址

SIMD和浮点寄存器V0-V31

- 专门用于向量运算和浮点运算，独立于通用寄存器
- 也称V寄存器，128位宽，应用程序可以通过多种形式访问
- Q0-Q31：128位
- D0-D31：64位
- S0-S31：32位
- H0-H31：16位
- B0-B31：8位





3.1.4 鲲鹏920处理器的架构及寻址

AArch64处理状态PSTATE

在AArch32执行状态下，当前处理器状态保存在当前程序状态寄存器CPSR中。

在AArch64执行状态下，当前处理器状态保存在名为PSTATE的数据结构中。

- PSTATE不是单一的寄存器，而是对处理器状态信息的抽象表征，处理信息被映射到多个寄存器中，可以通过指令访问这些寄存器。

- 可看做是保存当前处理状态信息的一组抽象寄存器或者一组标志信息的统称。



3.1.4 鲲鹏920处理器的架构及寻址

AArch64处理状态PSTATE

在EL0异常等级，可以访问的PSTATE信息包括条件标志和异常屏蔽位。

- 四个条件标志位：

负条件标志位N、零条件标志位Z、进位条件标志位C、溢出条件标志位V

- 四个异常屏蔽位，被置为1则屏蔽异常，置为0则允许异常处理：

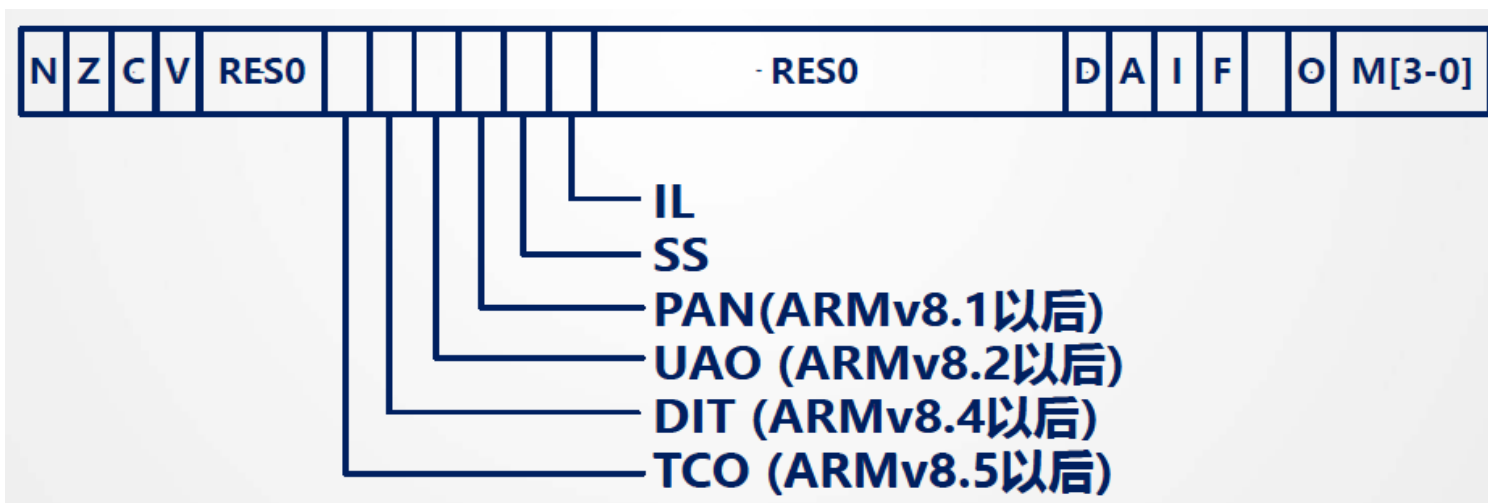
调试异常屏蔽位D、系统错误中断屏蔽位A、IRQ中断屏蔽位I、快速中断屏蔽位F。



3.1.4 鲲鹏920处理器的架构及寻址

备份程序状态寄存器

在AArch64状态异常发生时，当前的处理状态PSTATE参数被保存到备份程序状态寄存器SPSR中，用于在异常返回时恢复状态。





3.1.4 鲲鹏920处理器的架构及寻址

AArch64状态下的特殊功能寄存器





3.1.4 鲲鹏920处理器的架构及寻址

立即寻址

- 操作码字段后面的地址码部分就是操作数本身。
- 数据包含在指令当中，取指的同时取出了操作数。
 - `mov x0, #0xff00` ; 将操作数0xff00存入x0
; 操作数“0xff00”属于立即寻址
 - `sub x1, x1, #10` ; 将x1减10的值存入x1
; 操作数“#10”属于立即寻址



3.1.4 鲲鹏920处理器的架构及寻址

寄存器寻址

操作数的值在寄存器中，指令执行时直接从寄存器中取出操作数。

- `mov x0,x1` ; 将寄存器x1的值存入寄存器x0
- `sub x0,x1,x2` ; 将x1的值减去x2的值，结果存入寄存器x0



3.1.4 鲲鹏920处理器的架构及寻址

寄存器间接寻址

指令中给出的是寄存器编号，寄存器的内容是操作数在内存中的地址，操作数保存在该地址指向的单元中。

`ldr x0, [x1]` ; 将寄存器x1的数值作为地址，
; 取出此内存地址中的数据存入x0



3.1.4 鲲鹏920处理器的架构及寻址

基址寻址

- 将基址寄存器的内容与指令中给出的偏移量相加，形成操作数的有效地址。
- 用于访问基址附近的存储单元，常用于数组、查表操作。
- `ldr x0,[x1,#0x0f]` ; 将寄存器x1的数值加0x0f作地址，取出
; 此内存地址中的数据存入x0
- `str x0,[x1,#2]` ; 将寄存器x1的数值加2作为地址，将x0
; 中的数据存入此内存地址



3.1.4 鲲鹏920处理器的架构及寻址

堆栈寻址

使用一个专门的寄存器指向一块存储区域（堆栈指针指向堆栈）。指针所指向的存储单元为堆栈的栈顶。

- stmfd sp!, {x1-x7,lr} ; 将寄存器列表中寄存器的值存入堆栈
- ldmfd sp!, {x1-x7,lr} ; 将堆栈内容恢复到列表中的寄存器中



3.1.4 鲲鹏920处理器的架构及寻址

多寄存器寻址

一次可以传送多个寄存器的值，允许一条指令传送16个寄存器的任何子集或所有寄存器。

- `ldmia x0!, {x1-x7, x9}` ; 将x0地址中的值读出到列表寄存器中，
; 每次传送后x0自动加4
- `stmia x0!, {x1-x4, x6}` ; 将列表寄存器保存到x0指向的地址，
; 每次传送后x0自动加4



3.1.4 鲲鹏920处理器的架构及寻址

寄存器偏移寻址

在寄存器寻址的基础上，将寄存器中的数值移位后作为操作数。

- `ldr x0,x1,ls1 #2` ; `x1` 的值逻辑左移2 位，结果放入`x0`，即
; $x0 = x1 * 4$
- `ldr x0,x1,ror x2` ; 将`x1`的值循环右移`x2`中的值的对应位后，
; 写入`x0`